ВІННИЦЬКИЙ НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ

ІНСТИТУТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ ТА КОМП'ЮТЕРНОЇ ІНЖЕНЕРІЇ

КАФЕДРА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

Дослідження алгоритмів сортування

Звіт про виконання лабораторної роботи № 4

Виконав (ла): ст. 2 курсу, групи 2ПІ-25Б

Коваль Олександр

Перевірив: викладач Карась О.В.

Вінниця 2026

Тема: Дослідження алгоритмів сортування. Сортування вставкою та сортування деревом.

Мета: набути навичок реалізації та аналізу алгоритмів сортування мовою C++; експериментально порівняти час виконання двох алгоритмів на однакових вхідних масивах та перевірити відповідність отриманих результатів теоретичним оцінкам.

ЗАВДАННЯ

Програмно реалізувати два алгоритми сортування: № 3 — сортування вставкою (Insertion sort) та № 8 — сортування за допомогою двійкового дерева пошуку (Tree sort).

Порівняти між собою час їх виконання на однакових випадкових масивах різної розмірності.

Зробити висновок щодо відповідності отриманих результатів заявленим теоретичним оцінкам часу виконання.

КОД ПРОГРАМИ

```
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>

using namespace std;
using namespace chrono;







void insertionSort(vector<int>& a) {
    for (int i = 1; i < (int)a.size(); i++) {
        int key = a[i];
```

```
        int j = i;
        while (j > 0 && a[j - 1] > key) {
            a[j] = a[j - 1];
            j--;
        }
        a[j] = key;
    }
}



struct Node {
    int key;
    Node* left;
    Node* right;

    explicit Node(int k) : key(k), left(nullptr), right(nullptr) {}
};




Node* insert(Node* root, int key) {
    if (root == nullptr) return new Node(key);
    if (key < root->key)
        root->left  = insert(root->left, key);
    else
        root->right = insert(root->right, key);
    return root;
}
```

```
void inorder(Node* v, vector<int>& a, int& k) {
    if (v == nullptr) return;
    inorder(v->left, a, k);
    a[k++] = v->key;
    inorder(v->right, a, k);
}



void freeTree(Node* v) {
    if (v == nullptr) return;
    freeTree(v->left);
    freeTree(v->right);
    delete v;
}









void treeSort(vector<int>& a) {
    Node* root = nullptr;
    for (int x : a) root = insert(root, x);
    int k = 0;
    inorder(root, a, k);
    freeTree(root);
}
```

```
vector<int> makeArray(int n) {
    mt19937 gen(12345);
    uniform_int_distribution<int> dist(0, 1000000000);
    vector<int> a(n);
    for (int i = 0; i < n; i++) a[i] = dist(gen);
    return a;
}

int main() {
    cout << "n\tInsertion (ms)\tTree (ms)\n";

    vector<int> a1;
    vector<int> a2;

    for (int n : {10000, 100000}) {
        a1 = makeArray(n);
        a2 = a1;

        auto t1 = high_resolution_clock::now();
        insertionSort(a1);
        auto t2 = high_resolution_clock::now();
        treeSort(a2);
        auto t3 = high_resolution_clock::now();

        double ms1 = duration<double, milli>(t2 - t1).count();
        double ms2 = duration<double, milli>(t3 - t2).count();
```

```
        if (!is_sorted(a1.begin(), a1.end()) || !is_sorted(a2.begin(), a2.end())) {
            cout << "ПОМИЛКА: масив не відсортовано!" << endl;
            return 1;
        }

        cout << n << "\t" << ms1 << "\t\t" << ms2 << "\n";
    }

    cout << "\nПісля сортування вставкою (перші 20): ";
    for (int i = 0; i < 20 && i < (int)a1.size(); i++)
        cout << a1[i] << " ";
    cout << endl;

    cout << "\nПісля сортування деревом (перші 20): ";
    for (int i = 0; i < 20 && i < (int)a2.size(); i++)
        cout << a2[i] << " ";
    cout << endl;

    cout << "\nНатисніть Enter..." << flush;
    cin.get();

    return 0;
}
```

РЕЗУЛЬТАТИ ТЕСТУВАННЯ ТА РОБОТИ ПРОГРАМИ

Вимірювання проведено на одному комп'ютері, компілятор g++ 14.2, прапорець -O2. Для обох алгоритмів використано однакові випадкові масиви з однаковим зерном генератора, щоб забезпечити коректність порівняння. Час вимірюється функцією high_resolution_clock. Оскільки результат залежить від завантаження процесора, кожне вимірювання повторювалося 7 разів і наведено медіанне значення.

Отримані результати:

n = 10000 — сортування вставкою: близько 23 мс, сортування деревом: близько 3,8 мс;

n = 100000 — сортування вставкою: близько 2363 мс, сортування деревом: близько 91 мс.

Вивід програми:

```
n      Insertion (ms)      Tree (ms)
10000  23.1198             3.8352
100000 2363.41             90.5547

Після сортування вставкою (перші 20): 858 30925 44200 46131 46983 55667 63470 69971
93415 107251 117074 123641 126720 150300 158922 172740 174954 179152 225068 233684

Після сортування деревом (перші 20): 858 30925 44200 46131 46983 55667 63470 69971
93415 107251 117074 123641 126720 150300 158922 172740 174954 179152 225068 233684

Натисніть Enter...
```

БЛОК-СХЕМА АЛГОРИТМУ СОРТУВАННЯ ВСТАВКОЮ

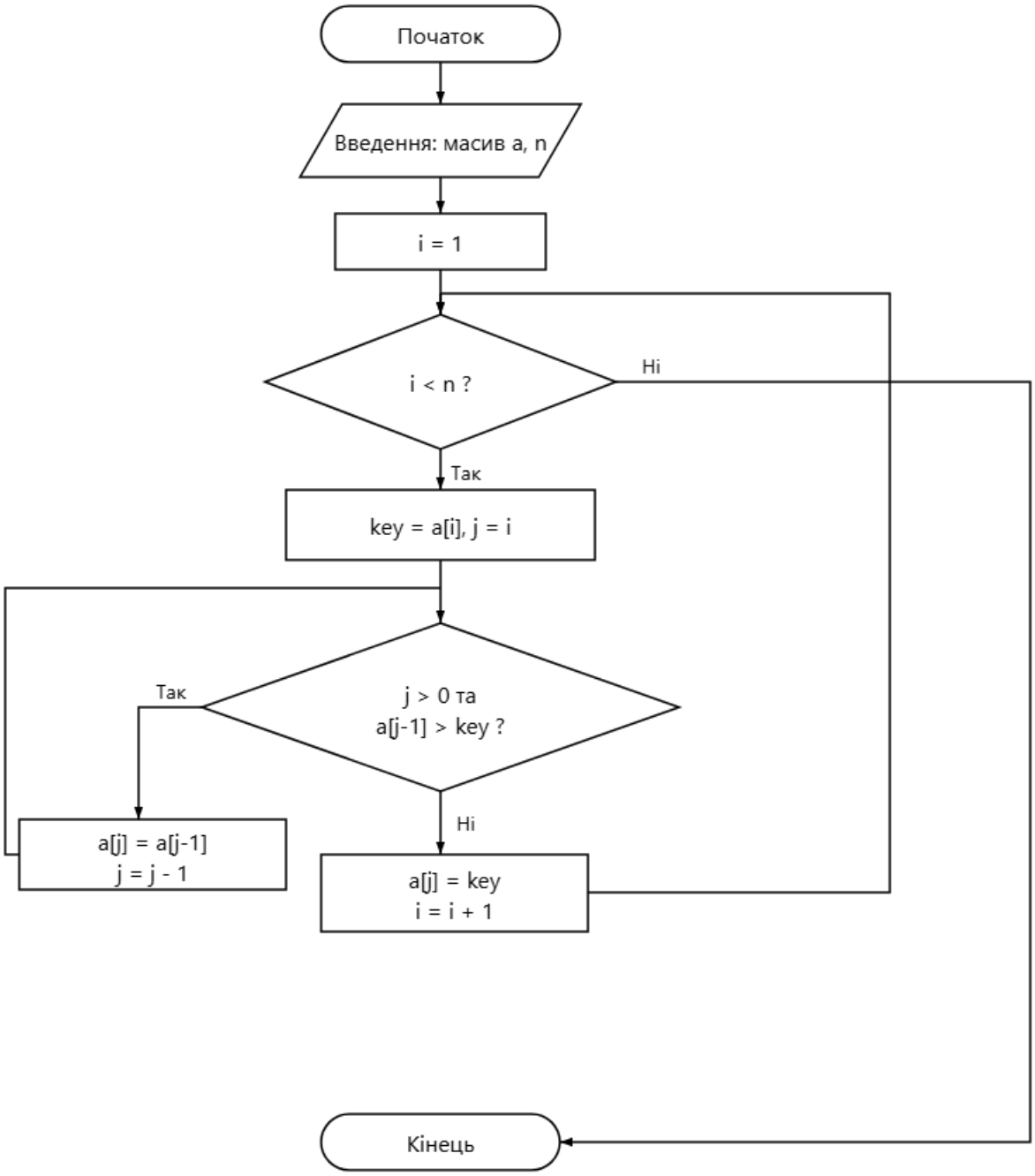


Рисунок 1 — Сортування вставкою

Вінниця 2026

БЛОК-СХЕМА АЛГОРИТМУ СОРТУВАННЯ ДЕРЕВОМ

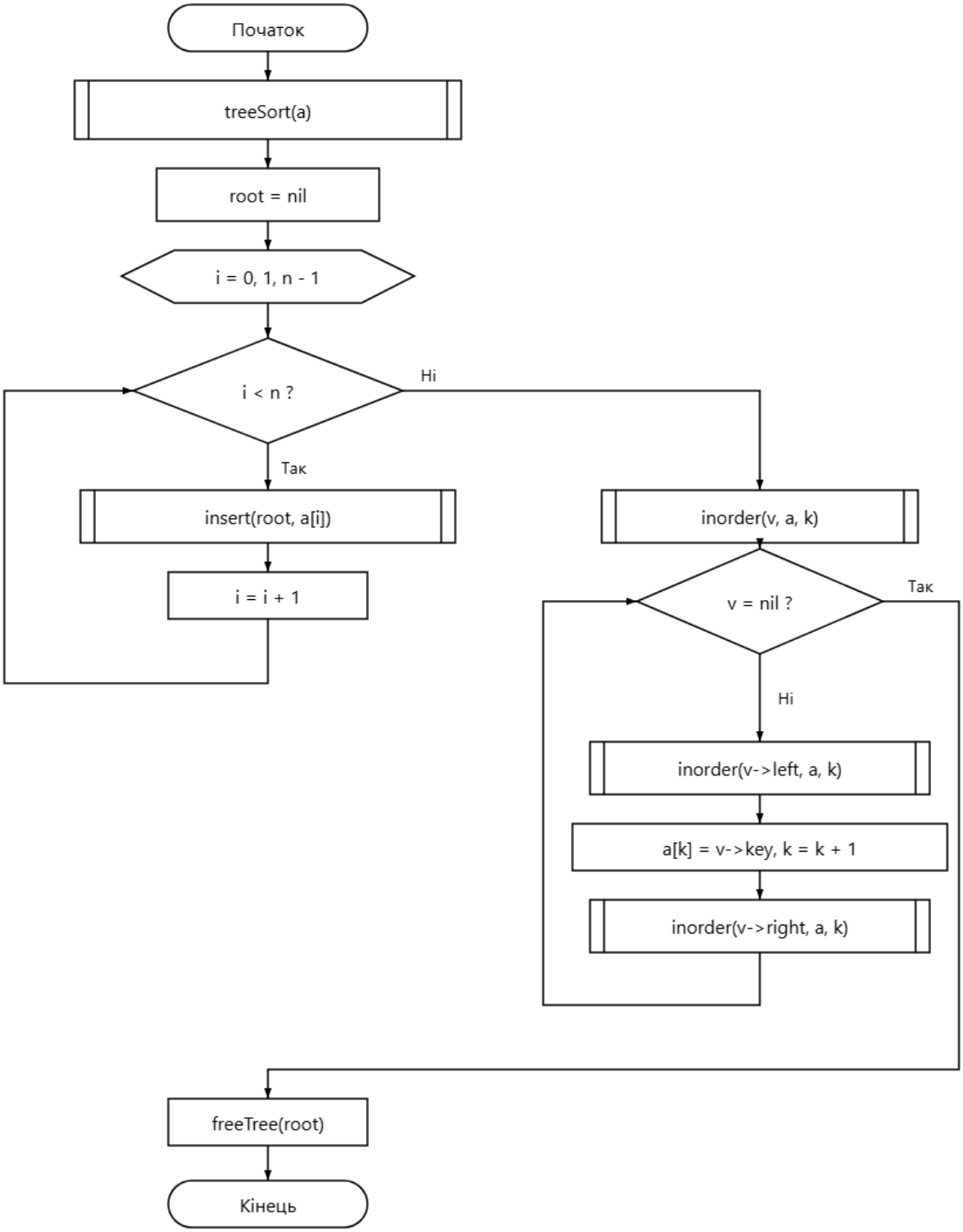


Рисунок 2 — Сортування деревом

Вінниця 2026

Висновок: У результаті виконання лабораторної роботи реалізовано два алгоритми сортування: сортування вставкою та сортування за допомогою двійкового дерева пошуку. Програма вимірює час виконання кожного алгоритму на однакових випадкових масивах та перевіряє коректність результату функцією is_sorted. Отримані результати відповідають заявленим теоретичним оцінкам: при збільшенні n у 10 разів час сортування вставкою зріс приблизно у 102 рази, що підтверджує оцінку $O(n^2)$, а час сортування деревом зріс приблизно у 24 рази, що відповідає оцінці $O(n \log n)$ у середньому випадку. Для $n \cdot \lg n$ теоретично очікується близько 12,5 разів; розбіжність спричинена не порівняннями, а витратами на виділення пам'яті для 100 000 вузлів дерева та промахами кешу під час обходу дерева. На n = 100000 сортування деревом виявилося приблизно у 26 разів швидшим за сортування вставкою. Слід зазначити, що найгірший випадок сортування деревом — $O(n^2)$: на вже відсортованому масиві дерево вироджується в ланцюг, а рекурсивний обхід — у переповнення стеку. Реалізація використовує лише стандартні бібліотеки iostream, vector, chrono, random та algorithm, прості функції та цикли; окремо перевірено збіжність результатів двох алгоритмів на однакових даних.

Вінниця 2026